

Week 14: Can We Predict Which Message Will Work?

Logistic Regression as One Neuron, Then the Multilayer Perceptron

Instructor / Mentor

Kiki Research Training Program

Week 14

Contents

1. From an Experiment to a Prediction Problem
2. Represent Messages with Numbers
3. Logistic Regression: One-Neuron Classification
4. From One Neuron to an MLP
5. Evaluate the Classifier
6. Python Demonstration
7. Research Judgment and Wrap-Up

From an Experiment to a Prediction Problem

Last Friday's Question Becomes Today's Dataset

Week 13 experiment

Does a new text message **cause** households to reduce electricity use from 5–8 p.m.?

Week 14 prediction problem

Suppose we have records from 100 message trials, including repeated templates tested under different conditions. Each row now has a label:

$$y = 1 \text{ useful}, \quad y = 0 \text{ not useful.}$$

Can we predict the label of a new message before paying for another large experiment?

Causal Question and Prediction Question Are Different

Experiment

Question: Did this message reduce use?

- treatment versus control;
- random assignment;
- estimated effect and uncertainty.

Classifier

Question: Which label is likely for a new message?

- input features;
- previously assigned labels;
- predicted probability.

Prediction does not prove causation

The classifier inherits the quality of the experiments used to create its labels.

How Could We Define “Useful”?

Before training any model, choose a rule without looking at the message features.

Illustrative labeling rule

Label a message useful when its randomized experiment estimates at least a meaningful reduction in average 5–8 p.m. use and meets the study’s evidence standard.

- The effect-size threshold should be chosen before inspecting results.
- Each test needs enough households; one response cannot label a message reliably.
- “Not useful” may mean ineffective, harmful, or simply too uncertain.
- Today we accept the labels and study the classification pipeline.

Week 14 Mission

By the end of class, students can:

1. turn message text into numeric features;
2. calculate a logistic-regression score and sigmoid probability;
3. explain why logistic regression acts like one artificial neuron;
4. explain how an MLP adds neurons and hidden layers;
5. interpret a confusion matrix, precision, recall, and F1-score; and
6. separate probability, classification, and causal evidence.

Scope boundary

We will say that gradient descent adjusts the weights, but we will not derive backpropagation or implement the training algorithm today.

120-Minute Flow

1. **0–15 min:** Week 13 experiment recap and the 100-message challenge.
2. **15–35 min:** labels, keyword features, and text embeddings.
3. **35–65 min:** logistic score, sigmoid probability, and thresholds.
4. **65–70 min:** break.
5. **70–90 min:** logistic regression as one neuron; MLP as many neurons.
6. **90–108 min:** evaluation, threshold choices, and common traps.
7. **108–116 min:** Python demonstration and prediction challenge.
8. **116–120 min:** exit ticket and homework briefing.

Represent Messages with Numbers

A Labeled Message Table

Message	Savings	Action	Deadline	Social	Useful
"Save \$3. Raise your thermostat before 5 p.m."	1	1	1	0	1
"Energy notice for this evening."	0	0	0	0	0
"Most neighbors are participating."	0	0	0	1	0
"Delay laundry until after 8 p.m."	0	1	0	0	0
"Save money. Delay laundry until after 8 p.m."	1	1	0	0	1

One row = one example

The input x contains features. The target y is the experimentally assigned label.

Route A: Human-Designed Features

A message can become a feature vector:

$$x = [\text{has savings} \quad \text{has action} \quad \text{has deadline} \quad \text{has social norm} \quad \text{length} \quad \text{send hour}]^T.$$

Example

“Save \$3. Delay laundry before 5 p.m.” might become

$$x = [1, 1, 1, 0, 40, 14]^T.$$

These features are understandable, inexpensive, and incomplete.

Categorical Features Must Become Numbers

Binary category

$$\text{has action} = \begin{cases} 1, & \text{yes,} \\ 0, & \text{no.} \end{cases}$$

Several unordered categories

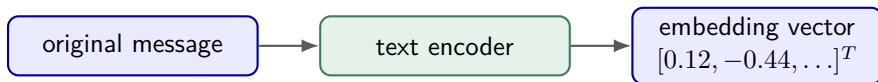
Tone = urgent, friendly, neutral can use one-hot features:

$[1, 0, 0]$, $[0, 1, 0]$, $[0, 0, 1]$.

Do not invent a false ranking

Coding urgent = 1, friendly = 2, neutral = 3 incorrectly suggests that neutral is “larger” than urgent.

Route B: Represent the Original Text



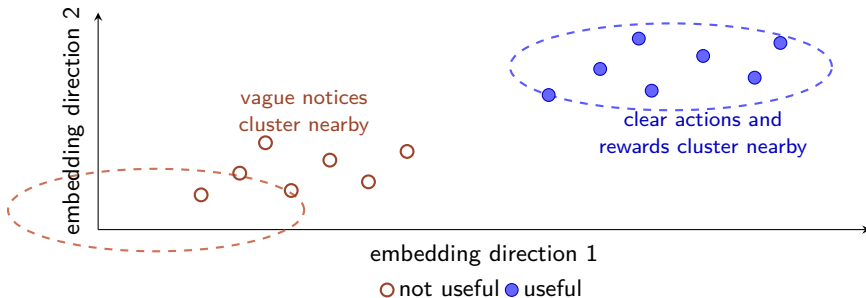
Text embedding

A text embedding is a numeric vector intended to preserve useful information about meaning and language similarity.

- It can capture more than a few chosen keywords.
- Its individual dimensions usually have no simple human meaning.
- The embedding is still just a numeric input to a classifier.

Embedding Space: Similar Messages Become Neighbors

A real embedding may have hundreds of dimensions. A two-dimensional projection helps us see the idea:



The axes are not literal topics

The projection is only a visual map. The classifier receives the full numeric vectors, not just these two displayed coordinates.

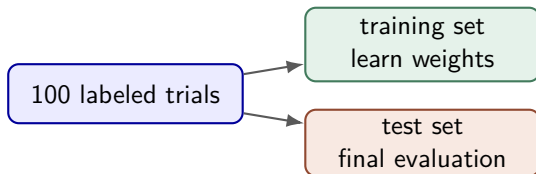
Keyword Flags, Text Vectors, or Both?

Representation	Strength	Limitation
Keyword / metadata features	interpretable; works with small data	misses wording and context
Bag-of-words or TF-IDF	uses the original vocabulary	weak semantic understanding
Text embedding	compact semantic representation	less interpretable; encoder matters
Combined features	mixes language and experiment metadata	more complexity and leakage risk

Leakage check

Never include the experimental outcome, p-value, or measured reduction as an input when predicting the useful label.

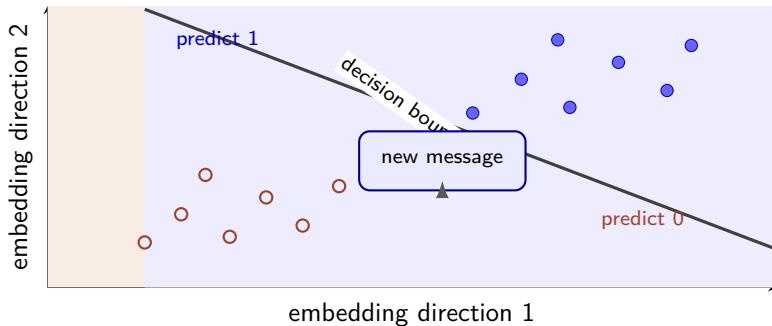
Split Before You Learn



- Training data are used to choose model parameters.
- Test data imitate genuinely unseen messages.
- Repeated near-duplicate messages should not be split across both groups.
- With only 100 examples, test results will have substantial uncertainty.

From an Embedding Cloud to a Decision Boundary

Training chooses a boundary using labeled examples; the boundary then divides the space for new messages.



The side of the boundary determines the class; distance from it influences the score and probability.

Logistic Regression: One-Neuron Classification

Step 1: Build a Weighted Score

Logistic regression first calculates

$$z = b + w_1x_1 + w_2x_2 + \cdots + w_dx_d = b + w^T x.$$

- x_j : an input feature.
- w_j : how strongly that feature changes the score.
- b : baseline score, called the bias or intercept.
- z : any real number, from $-\infty$ to $+\infty$.

Interpretation

Positive contributions push toward useful; negative contributions push toward not useful.

Hand Calculation: Score a New Message

Use four binary features:

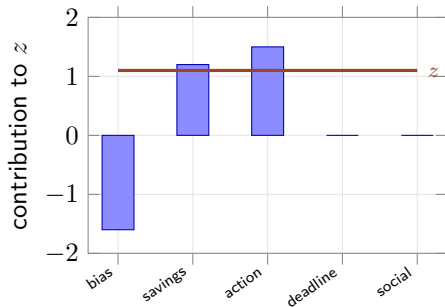
$$x = [\text{savings}, \text{action}, \text{deadline}, \text{social}]^T.$$

Suppose

$$w = [1.2, 1.5, 0.8, 0.4]^T, \quad b = -1.6.$$

For $x = [1, 1, 0, 0]^T$,

$$z = -1.6 + 1.2 + 1.5 + 0 + 0 = 1.1.$$



A score is not yet a probability

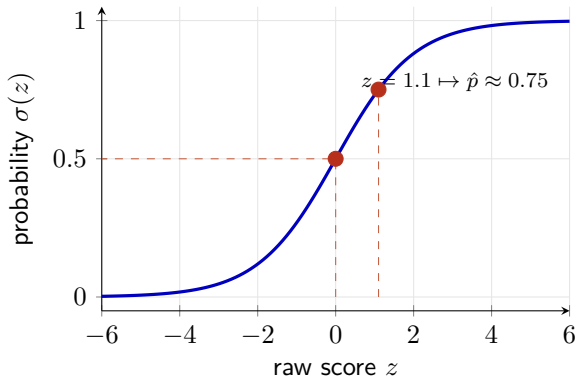
We need a function that maps every real score into the interval $(0, 1)$.

Step 2: The Sigmoid Function

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$

z	-4	-1	0	1	4
$\sigma(z)$	.02	.27	.50	.73	.98

- negative scores bend toward 0;
- positive scores bend toward 1;
- $z = 0$ maps to 0.5.



Why Sigmoid Fits the Probability Format

Before sigmoid

$$z = b + w^T x \in (-\infty, +\infty).$$

Values such as -2.7 and 4.1 cannot be probabilities.

After sigmoid

$$\hat{p} = \sigma(z) \in (0, 1).$$

We interpret $\hat{p}$ as the model's estimated probability of $y = 1$.

Probability quality must be checked

Producing a number between 0 and 1 does not guarantee that an 80% prediction succeeds 80% of the time.

Finish the Hand Calculation

We found $z = 1.1$. Therefore,

$$\hat{p} = \sigma(1.1) = \frac{1}{1 + e^{-1.1}} \approx 0.750.$$

Read the output carefully

The model assigns approximately 75% probability to the class “useful.” It does not claim the message will reduce every household’s use by 75%.

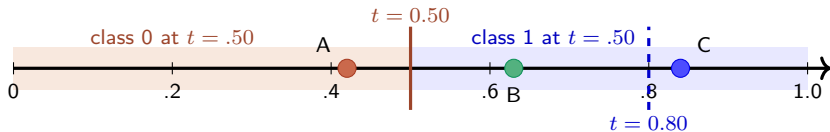
From probability to class

With threshold $t = 0.5$,

$$\hat{y} = \begin{cases} 1, & \hat{p} \geq 0.5, \\ 0, & \hat{p} < 0.5. \end{cases}$$

This message is classified as useful.

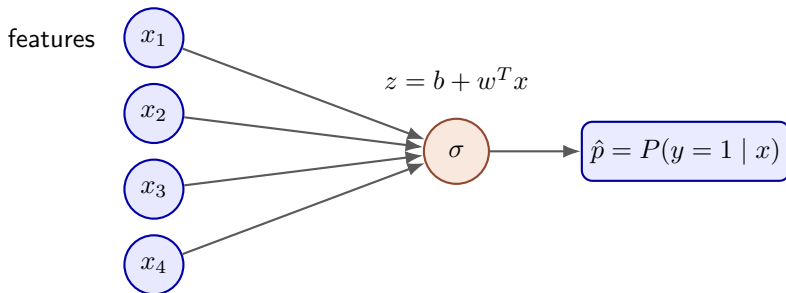
The Threshold Is a Decision, Not a Law



Message	Probability	$t = 0.50$	$t = 0.80$
A	0.42	not useful	not useful
B	0.63	useful	not useful
C	0.84	useful	useful

- A lower threshold flags more candidate messages.
- A higher threshold demands stronger model confidence.
- The appropriate threshold depends on the cost of testing a poor message versus missing a promising one.

Logistic Regression Is a Single Artificial Neuron



Useful bridge

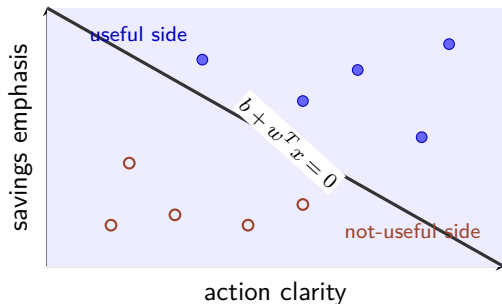
Logistic regression has weighted inputs, a bias, and an activation function—the same basic ingredients as one neuron with no hidden layer.

Its Boundary Is Linear in Feature Space

At threshold 0.5, the decision changes where

$$\sigma(z) = 0.5 \iff z = 0 \iff b + w^T x = 0.$$

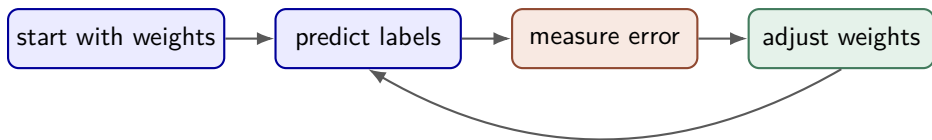
- two features: a line;
- three features: a plane;
- many features: a hyperplane.



Consequence

Logistic regression can combine many features, but the score is still one weighted sum. Some patterns need a more flexible model.

Where Do the Weights Come From?



High-level answer for today

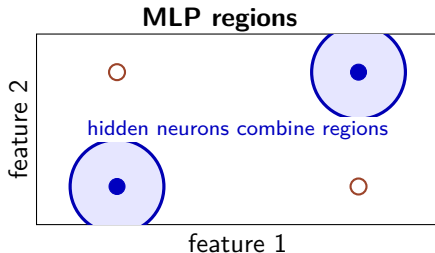
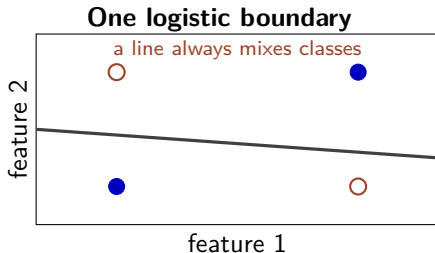
Training uses an optimization method such as gradient descent to adjust weights so predictions fit the training labels better.

We will call a library's `fit` method. The update equations and backpropagation are intentionally postponed.

From One Neuron to an MLP

Why Add More Neurons?

Suppose usefulness appears in separate feature combinations. One straight boundary may not separate them.



More flexible does not automatically mean better

With only 100 labeled messages, a larger network can memorize training examples and fail on new wording.

One Neuron Is Still the Basic Building Block

A neuron calculates

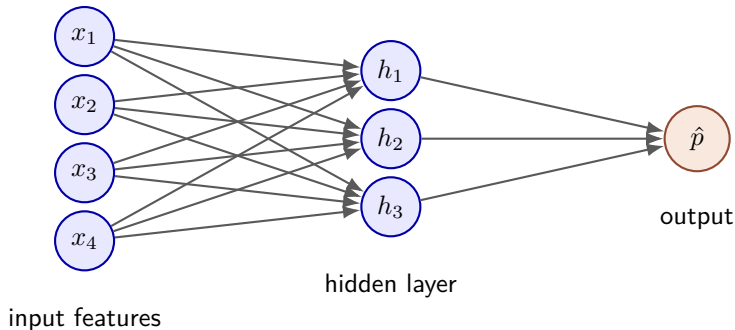
$$a = g \left(b + \sum_{j=1}^d w_j x_j \right).$$

- weighted sum: collect evidence from the inputs;
- bias: shift when the neuron activates;
- activation g : transform the score;
- output a : become an input to the next layer.

Activation choices

Hidden layers often use ReLU, $\text{ReLU}(z) = \max(0, z)$. A binary output often uses sigmoid.

A Multilayer Perceptron (MLP)



Multilayer perceptron

An MLP is a feed-forward network of connected neurons. Information moves from inputs through one or more hidden layers to the output.

Forward Pass: The Prediction Direction

For one hidden layer,

$$h = \text{ReLU}(W_1x + b_1), \quad \hat{p} = \sigma(W_2h + b_2).$$

1. Each hidden neuron receives all input features.
2. Hidden activations become a new representation of the message.
3. The output neuron combines the hidden activations.
4. Sigmoid converts the final score to a binary-class probability.

Forward pass is not training

A forward pass uses fixed weights to make a prediction. Training is the process that previously selected those weights.

Tiny Hidden-Layer Calculation

Let $x = [1, 1]^T$. One hidden neuron has $w = [0.8, -0.3]^T$ and $b = 0.1$.

$$z_h = 0.1 + 0.8(1) - 0.3(1) = 0.6,$$

$$h = \text{ReLU}(0.6) = 0.6.$$

A second hidden neuron can use different weights and detect a different pattern. The output neuron then combines both hidden activations.

What changed from logistic regression?

We inserted learned intermediate features between the original inputs and the final sigmoid neuron.

The Output Can Have More Than One Dimension

Task	Example output	Typical interpretation
Binary classification	one sigmoid value	useful versus not useful
Multiclass classification	K scores / probabilities	choose one of K message styles
Multilabel classification	K sigmoid values	several tags may all apply
Regression	one or more real values	predicted kWh reduction

Architecture follows the question

The number and type of output neurons depend on what the model must predict.

Logistic Regression and MLP Side by Side

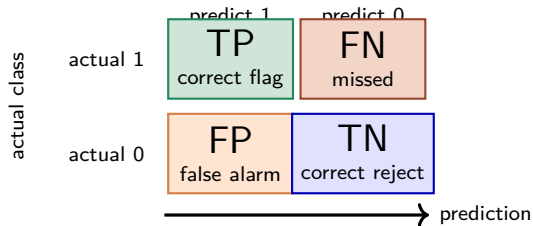
	Logistic regression	MLP
Hidden layers	none	one or more
Decision pattern	linear in input features	nonlinear and more flexible
Interpretability	coefficients are easier to inspect	distributed across many weights
Data need	often reasonable for small tabular data	usually benefits from more data
Risk	underfit complex interactions	overfit a small dataset
Binary output	sigmoid probability	usually sigmoid probability

Good research habit

Start with a simple baseline. A more complicated model must earn its complexity on unseen data.

Evaluate the Classifier

A Confusion Matrix Counts Four Outcomes



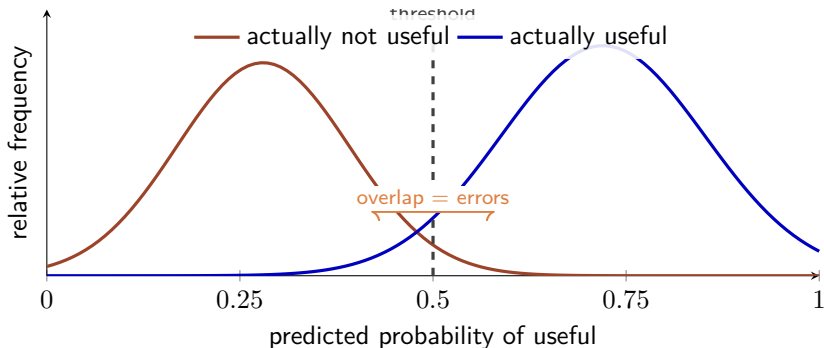
- **TP:** promising message correctly flagged.
- **FP:** poor message sent to another experiment.
- **FN:** promising message missed.
- **TN:** poor message correctly rejected.

Diagonal = correct

TP and TN are correct decisions; FP and FN are different kinds of error.

Predicted Probabilities Form Overlapping Distributions

A classifier rarely places every negative near 0 and every positive near 1.



Moving the threshold changes how much of each distribution becomes a positive prediction.

Accuracy, Precision, Recall, and F1

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}$$

$$F1 = 2 \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}.$$

Read them as questions

Precision: Of messages we flagged, how many were useful? Recall: Of all useful messages, how many did we find?

Metric Choice Depends on the Decision

False positives are costly

If every candidate requires an expensive field trial, prioritize precision so the short list is reliable.

False negatives are costly

If screening is cheap and missing a strong message is costly, prioritize recall so few promising messages are lost.

No metric is automatically correct

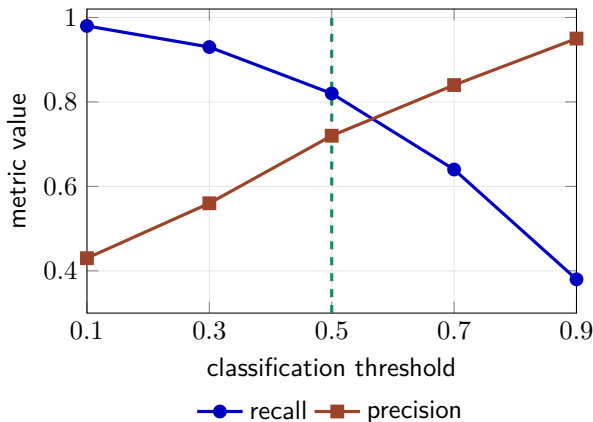
State who pays for each error and what action follows a positive prediction.

Thresholds Trade Precision Against Recall

- Lower threshold: flag more messages, usually increasing recall and false positives.
- Higher threshold: flag fewer messages, usually increasing precision and false negatives.

Choose an operating point

Use validation data and real decision costs, not the final test set.



Accuracy Can Hide Failure

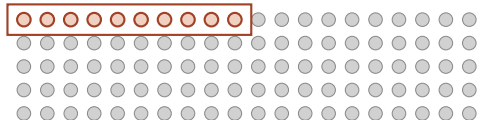
Suppose only 10 of 100 messages are useful. A model that always says “not useful” gets

$$\text{Accuracy} = \frac{90}{100} = 90\%.$$

Yet its recall is

$$\text{Recall} = \frac{0}{10} = 0.$$

10 useful messages: all missed



model predicts “not useful” for every dot

Always compare with a baseline

Ask whether the model beats simple rules such as predicting the majority class or flagging every message containing a clear action.

Small Data Makes Evaluation Noisy

If a 20-message test set contains 12 correct predictions, accuracy is 60%. One additional correct prediction changes it to 65%.

- Report the number of test examples, not only a percentage.
- Consider repeated cross-validation for model comparison.
- Keep a final untouched test set when enough data exist.
- Collect more diverse messages before trusting narrow differences.

Scientific conclusion

“MLP scored 5 points higher” is weak evidence if the test set is tiny or contains repeated templates.

Python Demonstration

Load the Synthetic Teaching Dataset

```
1 import pandas as pd
2
3 df = pd.read_csv("week14_messages.csv")
4
5 feature_cols = [
6     "has_savings", "has_action", "has_deadline",
7     "has_social_norm", "message_length", "send_hour"
8 ]
9
10 X = df[feature_cols]
11 y = df["useful"]
12
13 print(df[["text", "useful"]].head())
14 print(y.value_counts())
```

Teaching-data warning

This file is synthetic and simplified. Its scores do not demonstrate that these message styles work in the real world.

Prepare the Train–Test Split

```
1 from sklearn.model_selection import train_test_split
2 from sklearn.pipeline import make_pipeline
3 from sklearn.preprocessing import StandardScaler
4 from sklearn.linear_model import LogisticRegression
5 from sklearn.neural_network import MLPClassifier
6
7 X_train, X_test, y_train, y_test = train_test_split(
8     X, y, test_size=0.20, random_state=14, stratify=y
9 )
```

Why stratify=y?

It keeps approximately the same useful/not-useful class balance in the training and test sets.

Fit a Logistic Baseline and a Small MLP

```
1 logistic = make_pipeline(  
2     StandardScaler(), LogisticRegression(max_iter=2000)  
3 )  
4 mlp = make_pipeline(  
5     StandardScaler(),  
6     MLPClassifier(hidden_layer_sizes=(6,), max_iter=3000,  
7                   random_state=14)  
8 )  
9  
10 logistic.fit(X_train, y_train)  
11 mlp.fit(X_train, y_train)
```

What `fit` hides today

Both models optimize their weights from labeled training examples. We use the library without deriving the update procedure.

Compare Unseen Predictions

```
1 from sklearn.metrics import confusion_matrix, classification_report
2
3 for name, model in [("logistic", logistic), ("MLP", mlp)]:
4     probability = model.predict_proba(X_test)[: , 1]
5     prediction = (probability >= 0.50).astype(int)
6
7     print("\n", name)
8     print(confusion_matrix(y_test, prediction))
9     print(classification_report(y_test, prediction, digits=3))
```

The comparison question

Does the MLP improve unseen-message performance enough to justify less interpretability and greater overfitting risk?

Using Precomputed Text Embeddings

If an approved text encoder has already produced one vector per message:

```
1 # embeddings has shape (number_of_messages, embedding_dimension)
2 embeddings = load_precomputed_embeddings()
3
4 X_train, X_test, y_train, y_test = train_test_split(
5     embeddings, y, test_size=0.20,
6     random_state=14, stratify=y
7 )
8
9 model = LogisticRegression(max_iter=2000)
10 model.fit(X_train, y_train)
11 probability = model.predict_proba(X_test)[: , 1]
```

Same classifier, different representation

Logistic regression can accept keyword features, TF-IDF vectors, embeddings, or a combination. The input dimension changes; the core score-sigmoid idea does not.

Prediction Challenge

Consider three new messages:

1. “Please conserve energy tonight.”
2. “Save \$4: raise your thermostat two degrees from 5–8 p.m.”
3. “Most nearby households joined. Delay laundry until after 8 p.m.”

In groups:

1. encode four binary features for each message;
2. predict which message logistic regression will favor;
3. name one meaning an embedding may capture that the four flags miss; and
4. state which message you would test next and why.

Research Judgment and Wrap-Up

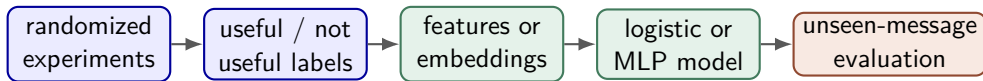
What Could Go Wrong in the Real Study?

- **Bad labels:** experiments were too small or used inconsistent definitions of useful.
- **Selection bias:** the 100 messages are not representative of future messages.
- **Template leakage:** nearly identical messages appear in training and test sets.
- **Population shift:** a message tested in summer may behave differently in winter.
- **Fairness and accessibility:** language, income, disability, and housing constraints affect who can respond.
- **Over-automation:** the prediction replaces rather than prioritizes controlled testing.

Recommended role

Use the classifier to rank candidates for careful experiments, not as a substitute for experiments.

The Complete Map



Deployment loop

Predictions nominate new candidates; new controlled experiments create better evidence and eventually better labels.

Exit Ticket

Answer in one sentence or one calculation each:

1. What is the difference between proving a message caused a reduction and predicting its label?
2. Calculate z for $b = -1$, $w = [1, 2]^T$, and $x = [1, 0]^T$.
3. Why is sigmoid useful for binary classification?
4. In what sense is logistic regression a one-neuron network?
5. What does a hidden layer add?
6. When might recall matter more than precision in this project?

Homework for This Week

Manual programming task

Complete `week14_message_classifier.py`: logistic score, sigmoid, thresholding, confusion-matrix metrics, and a small MLP forward pass.

Library modeling task

Use the supplied synthetic `week14_messages.csv` to compare logistic regression and a small MLP. You may call `fit`; you do not implement gradient descent.

Research explanation

Explain why the classifier can prioritize future experiments but cannot establish that a new message causes lower electricity use.

Key Takeaways

1. Classification starts with numeric inputs and trustworthy labels.
2. Logistic regression uses a weighted score and sigmoid to estimate a binary probability.
3. Logistic regression can be viewed as one sigmoid neuron with no hidden layer.
4. An MLP adds neurons and hidden layers, allowing more flexible nonlinear patterns.
5. Thresholds and evaluation metrics must match the real decision costs.
6. Prediction should guide the next experiment, not replace causal evidence.